Modül 8

Routing & Navigation

Sayfa Yönetimi ve Navigasyon

Modül:	8 / 30
Ders Sayısı:	9 ders
Seviye:	Orta
Versiyon:	Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

🚀 Modül 8'e Hoş Geldin

Bu modülde Angular'ın routing sistemini detaylı işleyeceğiz. Sayfa geçişleri, parametreler, lazy loading, guards, resolvers — hepsi.

Bu modülün sonunda yapabileceklerin:

- ✓ Route configuration ve hierarchical routes
- ✓ Route parameters (path params, query params)
- ✓ Navigation programatik ve template ile
- ✓ Lazy loading routes
- ✓ Guards ile route koruması (CanActivate, vs.)
- ✓ Resolvers ile data preloading
- ✓ Child routes ve nested routing
- ✓ Route data ve title
- ✓ Modern functional API'lar (Angular 19+)

DERS 8.1

Route Configuration — Temel

Angular'da route'lar nasıl tanımlanır?

🗺️ Temel Route Yapısı

```
// app.routes.ts
import { Routes } from '@angular/router';

export const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'users', component: UsersComponent },
  { path: 'users/:id', component: UserDetailComponent },
  { path: 'about', component: AboutComponent },

  // Wildcard - sonda
  { path: '**', component: NotFoundComponent },
];
```

⚙️ Routing'i Kaydetme

```
// app.config.ts
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
  ]
};
```

📍 router-outlet

`router-outlet`: Aktif route'un component'i burada render edilir.

```
<!-- app.html -->
<nav>
  <a routerLink="/">Home</a>
  <a routerLink="/users">Users</a>
  <a routerLink="/about">About</a>
</nav>

<main>
  <router-outlet />  <!-- Aktif route component'i burada -->
</main>
```

routerLink

```
<!-- Static URL -->
<a routerLink="/users">Users</a>

<!-- Dynamic URL -->
<a [routerLink]="['/users', userId]">User Detail</a>

<!-- Query params -->
<a [routerLink]="['/search']"
   [queryParams]="{ q: 'angular', page: 1 }">
  Search
</a>

<!-- Fragment -->
<a [routerLink]="['/docs']" fragment="section-1">
  Docs Section 1
</a>

<!-- Active class -->
<a routerLink="/users"
   routerLinkActive="active"
   [routerLinkActiveOptions]="{ exact: true }">
  Users
</a>
```

DERS 8.2

Route Parameters

URL'den parametre okumak: path params, query params, fragment.

Path Parameters

```
// Route tanımı
{ path: 'users/:id', component: UserDetailComponent }
{ path: 'posts/:slug/comments/:commentId', component: CommentComponent }
```

Component'te Okuma:

```
// Modern: input.required() ile (v16+)
@Component({...})
export class UserDetailComponent {
  // Otomatik route paramından çekilir
  id = input.required<string>();  // → /users/:id

  constructor() {
    effect(() => {
      console.log('User ID:', this.id());
    });
  }
}

// app.config.ts'te aktif et:
provideRouter(routes, withComponentInputBinding())
```

Alternatif (eski): ActivatedRoute

```
@Component({...})
export class UserDetailComponent {
  private route = inject(ActivatedRoute);

  // Observable approach
  id$ = this.route.paramMap.pipe(
    map(params => params.get('id'))
  );

  // Signal approach
  id = toSignal(
    this.route.paramMap.pipe(map(p => p.get('id'))),
    { initialValue: null }
  );
}
```

Query Parameters

```
// URL: /search?q=angular&page=2

@Component({...})
export class SearchComponent {
  private route = inject(ActivatedRoute);

  // Observable
  query$ = this.route.queryParamMap.pipe(
    map(p => ({
      q: p.get('q') || '',
      page: Number(p.get('page')) || 1
    }))
  );

  // Signal
  searchParams = toSignal(this.query$, {
    initialValue: { q: '', page: 1 }
  });
}
```

#□ Fragment

```
// URL: /docs#installation

@Component({...})
export class DocsComponent {
  private route = inject(ActivatedRoute);

  fragment = toSignal(this.route.fragment, { initialValue: null });

  constructor() {
    effect(() => {
      const frag = this.fragment();
      if (frag) {
        document.getElementById(frag)?.scrollIntoView();
      }
    });
  }
}
```

DERS 8.3

Programmatic Navigation

Code'dan navigation yapmak: Router service.

Router.navigate()

```
@Component({...})
export class MyComponent {
  private router = inject(Router);

  // Basit navigation
  goToHome() {
    this.router.navigate(['/']);
  }

  // Path params ile
  goToUser(id: string) {
    this.router.navigate(['/users', id]);
  }

  // Query params ile
  search(query: string) {
    this.router.navigate(['/search'], {
      queryParams: { q: query, page: 1 }
    });
  }

  // Mevcut query params'ı koru
  changeFilter(filter: string) {
    this.router.navigate([], {
      queryParams: { filter },
      queryParamsHandling: 'merge'  // veya 'preserve'
    });
  }

  // Relative navigation
  goToChild() {
    this.router.navigate(['./detail'], { relativeTo: this.route });
  }

  // Fragment ile
  goToSection(section: string) {
    this.router.navigate(['/docs'], { fragment: section });
  }
}
```

navigateByUrl()

```
this.router.navigateByUrl('/users/123');
this.router.navigateByUrl('/search?q=angular');

// Tek string URL ile gitmek için kolaylık
```

Location Service

```
import { Location } from '@angular/common';

@Component({...})
export class MyComponent {
  private location = inject(Location);

  goBack() {
    this.location.back();
  }

  goForward() {
    this.location.forward();
  }

  // URL'i değiştir ama route'u tetikleme
  replaceState(url: string) {
    this.location.replaceState(url);
  }
}
```

DERS 8.4

Lazy Loading

Route'ları sadece gerektiğinde yüklemek — performans için kritik.

Component-Level Lazy Loading (Modern)

```
export const routes: Routes = [
  {
    path: 'home',
    component: HomeComponent  // Eager loaded
  },
  {
    path: 'admin',
    loadComponent: () => import('./admin/admin.component')
      .then(m => m.AdminComponent)
    // Bu sadece /admin'e gidilince yüklenir
  },
  {
    path: 'reports',
    loadComponent: () => import('./reports/reports.component')
      .then(m => m.ReportsComponent)
  }
];
```

Feature-Level Lazy Loading

```
// Birden fazla route'u birlikte yükle
export const routes: Routes = [
  {
    path: 'admin',
    loadChildren: () => import('./admin/admin.routes')
      .then(m => m.ADMIN_ROUTES),
    // Provider'lar bu feature için
    providers: [AdminService, AdminAuthGuard]
  }
];

// admin/admin.routes.ts
export const ADMIN_ROUTES: Routes = [
  { path: '', component: AdminDashboardComponent },
  { path: 'users', component: AdminUsersComponent },
  { path: 'settings', component: AdminSettingsComponent },
];
```

⚡ Preloading Strategies

Lazy routes'ları arka planda önceden indir:

```
// app.config.ts
import { provideRouter, withPreloading, PreloadAllModules } from '@angular/router';

provideRouter(
  routes,
  withPreloading(PreloadAllModules)  // Hepsini önceden indir
)

// Custom strategy
@Injectable({ providedIn: 'root' })
export class SelectivePreloader implements PreloadingStrategy {
  preload(route: Route, load: () => Observable<any>): Observable<any> {
    return route.data?.['preload'] ? load() : of(null);
  }
}

// Route'larda işaretle
{ path: 'admin', loadComponent: ..., data: { preload: true } }
```

DERS 8.5

Route Guards — Modern Functional API

Bir route'a erişimi kontrol etmek. Functional guards modern yöntem.

Guard Türleri

Guard	Ne Zaman?	Use Case
CanActivate	Route'a girmeden önce	Auth check, permission
CanDeactivate	Route'tan çıkmadan önce	Unsaved changes warn
CanMatch	Route match olmadan	Lazy load filter
CanLoad (deprecated)	Lazy module yüklenmeden	CanMatch tercih edin
Resolve	Route'a girmeden data fetch	Pre-load data

Modern: Functional Guards

```
// guards/auth-guard.ts
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../services/auth.service';

export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  if (auth.isLoggedIn()) {
    return true;
  }

  // Yönlendir
  router.navigate(['/login'], {
    queryParams: { returnUrl: state.url }
  });
  return false;
};

// Route'a uygulama
{
  path: 'admin',
  loadComponent: () => import('./admin/admin.component'),
  canActivate: [authGuard]
}
```

🔐 Role-Based Guard

```
export const adminGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  const user = auth.currentUser();
  if (user?.role === 'admin') {
    return true;
  }

  router.navigate(['/forbidden']);
  return false;
};

// Parametreli guard (factory)
export function roleGuard(allowedRoles: string[]): CanActivateFn {
  return () => {
    const auth = inject(AuthService);
    const user = auth.currentUser();
    return user ? allowedRoles.includes(user.role) : false;
  };
}

// Kullanım
{ path: 'admin', canActivate: [roleGuard(['admin', 'superadmin'])] }
```

⚠️ CanDeactivate — Çıkış Onayı

```
// Component'in bir method'u olmalı
export interface CanComponentDeactivate {
  canDeactivate: () => boolean | Promise<boolean>;
}

// Guard
export const unsavedChangesGuard: CanDeactivateFn<CanComponentDeactivate> =
(component) => {
  return component.canDeactivate();
};

// Component
@Component({...})
export class EditFormComponent implements CanComponentDeactivate {
  hasChanges = signal(false);

  canDeactivate() {
    if (this.hasChanges()) {
      return confirm('Kaydedilmemiş değişiklikler var, çıkmak istiyor musunuz?');
    }
    return true;
  }
}

// Route
{ path: 'edit', component: EditFormComponent, canDeactivate: [unsavedChangesGuard] }
```

DERS 8.6

Resolvers — Pre-fetch Data

Route'a girmeden önce data'yı önceden yükle.

Resolver Nedir?

Component yüklemeden önce HTTP isteği yapıp data'yı hazır et. Component yüklendiğinde data zaten var.

Functional Resolver

```
// resolvers/user-resolver.ts
import { ResolveFn } from '@angular/router';
import { inject } from '@angular/core';

export const userResolver: ResolveFn<User> = (route) => {
  const userService = inject(UserService);
  const id = route.paramMap.get('id')!;
  return userService.getUser(id);  // Observable veya Promise
};

// Route
{
  path: 'users/:id',
  component: UserDetailComponent,
  resolve: { user: userResolver }
}
```

Component'te Kullanım

```
@Component({...})
export class UserDetailComponent {
  // Modern: Input ile
  user = input.required<User>();  // Route data'dan otomatik

  // Veya ActivatedRoute ile
  private route = inject(ActivatedRoute);
  user2$ = this.route.data.pipe(map(d => d['user']));
}

// app.config.ts:
provideRouter(routes, withComponentInputBinding())
```

DERS 8.7

Child Routes & Nested Routing

Route hiyerarşileri ve nested layouts.

Child Routes

```
export const routes: Routes = [
  {
    path: 'users',
    component: UsersLayoutComponent,  // Parent
    children: [
      { path: '', component: UserListComponent },
      { path: ':id', component: UserDetailComponent },
      { path: ':id/edit', component: UserEditComponent },
    ]
  }
];
```

Parent'ta router-outlet:

```
<!-- users-layout.component.html -->
<aside class="sidebar">
  <h2>Users</h2>
  <nav>
    <a routerLink="/users">All Users</a>
    <a routerLink="/users/new">Create</a>
  </nav>
</aside>

<main>
  <router-outlet />  <!-- Child component buraya -->
</main>
```

Named Outlets (Secondary Routes)

Birden fazla outlet — modal, popup, sidebar için:

```
// Routes
{ path: '', component: HomeComponent },
{ path: 'help', component: HelpComponent, outlet: 'sidebar' }

// Template
<router-outlet />
<router-outlet name="sidebar" />

// Navigation
this.router.navigate([{ outlets: { sidebar: ['help'] } }]);
// URL: /(sidebar:help)

// Close
this.router.navigate([{ outlets: { sidebar: null } }]);
```

DERS 8.8

Route Data, Title & Metadata

Route'lara ek bilgi ekleme.

Route Title (v16+)

```
// Static title
{ path: 'home', component: HomeComponent, title: 'Ana Sayfa' }

// Dynamic title (function)
{
  path: 'users/:id',
  component: UserDetailComponent,
  title: (route) => {
    const userService = inject(UserService);
    const id = route.paramMap.get('id')!;
    return userService.getUser(id).then(u => u.name);
  }
}

// Title strategy (custom)
@Injectable({ providedIn: 'root' })
export class CustomTitleStrategy extends TitleStrategy {
  override updateTitle(snapshot: RouterStateSnapshot) {
    const title = this.buildTitle(snapshot);
    if (title) {
      document.title = `${title} | My App`;
    }
  }
}

// Register
provideRouter(routes, { titleStrategy: CustomTitleStrategy })
```

Route Data

```
{
  path: 'admin',
  component: AdminComponent,
  data: {
    requiresAuth: true,
    role: 'admin',
    breadcrumb: 'Yönetim Paneli'
  }
}

// Component'te
@Component({...})
export class AdminComponent {
  private route = inject(ActivatedRoute);

  data = toSignal(this.route.data, { initialValue: {} as Data });
}
```

DERS 8.9

Advanced Patterns

Production projelerinde sık görülen pattern'ler.

Pattern 1: Auth + Return URL

```
export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  if (auth.isLoggedIn()) return true;

  // Return URL ile login'e yönlendir
  router.navigate(['/login'], {
    queryParams: { returnUrl: state.url }
  });
  return false;
};

// LoginComponent'te
@Component({...})
export class LoginComponent {
  private router = inject(Router);
  private route = inject(ActivatedRoute);

  onLoginSuccess() {
    const returnUrl = this.route.snapshot.queryParamMap.get('returnUrl') || '/';
    this.router.navigateByUrl(returnUrl);
  }
}
```

Pattern 2: Breadcrumb

```
// Route'larda data
{
  path: 'users',
  data: { breadcrumb: 'Users' },
  children: [
    { path: ':id', data: { breadcrumb: (route: any) => `User ${route.params.id}` } }
  ]
}

// Breadcrumb service
@Injectable({ providedIn: 'root' })
export class BreadcrumbService {
  private router = inject(Router);

  breadcrumbs = toSignal(
    this.router.events.pipe(
      filter(e => e instanceof NavigationEnd),
      map(() => this.buildBreadcrumbs(this.router.routerState.root))
    ),
    { initialValue: [] }
  );

  private buildBreadcrumbs(route: ActivatedRoute, breadcrumbs: any[] = []): any[] {
    const children = route.children;
    if (children.length === 0) return breadcrumbs;

    for (const child of children) {
      const routeURL = child.snapshot.url.map(s => s.path).join('/');
      const label = child.snapshot.data['breadcrumb'];
      if (label) {
        breadcrumbs.push({ label, url: routeURL });
      }
      return this.buildBreadcrumbs(child, breadcrumbs);
    }
    return breadcrumbs;
  }
}
```

□ Pattern 3: Scroll to Top

```
// app.config.ts
provideRouter(
  routes,
  withInMemoryScrolling({
    scrollPositionRestoration: 'top',  // Her route'a top'tan başla
    anchorScrolling: 'enabled'         // Fragment'a scroll
  })
)
```

□ Pattern 4: Route Animations

```
// Animation trigger
const routeAnimations = trigger('routeAnimations', [
  transition('* <=> *', [
    style({ opacity: 0 }),
    animate('300ms', style({ opacity: 1 }))
  ])
]);

// Component
@Component({
  template: `
    <div [@routeAnimations]="getRouteAnimation(outlet)">
      <router-outlet #outlet="outlet" />
    </div>
  `,
  animations: [routeAnimations]
})
export class App {
  getRouteAnimation(outlet: RouterOutlet) {
    return outlet?.activatedRouteData?.['animation'] || 'default';
  }
}
```

Modül 8 Özeti

Neler Öğrendin?

- ✓ Route configuration ve hierarchy
- ✓ Path params, query params, fragments
- ✓ Programmatic navigation (Router.navigate)
- ✓ Lazy loading components ve features
- ✓ Functional guards (CanActivate, CanDeactivate)
- ✓ Resolvers ile pre-fetch data
- ✓ Child routes ve named outlets
- ✓ Route title ve data
- ✓ Production pattern'leri (auth flow, breadcrumb, scroll)

Sıradaki Modül

Modül 9 — Forms. Template-driven forms, Reactive forms, Signal Forms (preview), validation, custom validators, dynamic forms.